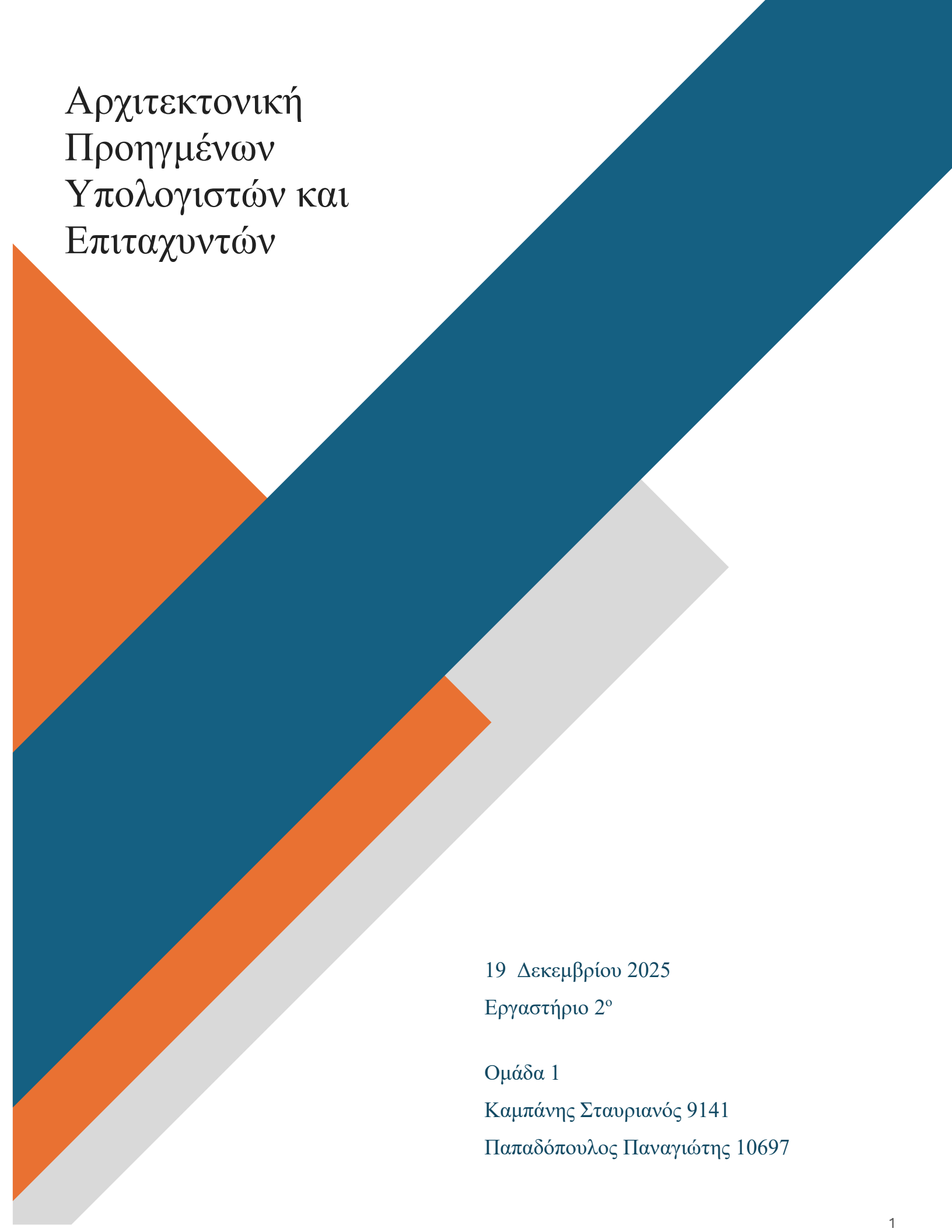




Αρχιτεκτονική Προηγμένων Υπολογιστών και Επιταχυντών

19 Δεκεμβρίου 2025

Εργαστήριο 2°

Ομάδα 1

Καμπάνης Σταυριανός 9141

Παπαδόπουλος Παναγιώτης 10697

1. Εισαγωγή

Σκοπός του παρόντος εργαστηρίου ήταν η εξοικείωση με την ενοποιημένη πλατφόρμα Vitis της Xilinx και το μοντέλο εκτέλεσης Host-Kernel. Η διαδικασία περιελάμβανε την εκτέλεση ενός εισαγωγικού παραδείγματος (vadd) και στη συνέχεια την ανάπτυξη, ενσωμάτωση και επαλήθευση ενός επιταχυντή επεξεργασίας εικόνας (image_diff_sharpen) στην πλατφόρμα Alveo U200.

Η ροή εργασίας εστίασε στη διασύνδεση του κώδικα Host (OpenCL) με τον πυρήνα (C++ Kernel), τη διαχείριση μνήμης και την εκτέλεση σε περιβάλλοντα προσομοίωσης (Software & Hardware Emulation).

2. Μέρος A: Εισαγωγή στο Vitis (vadd)

Στο πρώτο μέρος, μελετήθηκε και εκτελέστηκε το παράδειγμα vector addition (vadd). Μέσω αυτού κατανοήσαμε τη δομή ενός Vitis project, τη διαδικασία μεταγλώττισης για Software και Hardware Emulation και τη χρήση του Vitis Analyzer για την επισκόπηση της απόδοσης και των μεταφορών δεδομένων.

3. Μέρος B: Σχεδίαση H/W Accelerator (IMAGE_DIFF_SHARPEN)

Στο δεύτερο μέρος, ο επιταχυντής του Εργαστηρίου 1 επεκτάθηκε ώστε να περιλαμβάνει ένα επιπλέον στάδιο φίλτρου οξύτητας (Sharpen Filter 3×3). Ο επιταχυντής δέχεται δύο εικόνες εισόδου, υπολογίζει τη διαφορά τους, εφαρμόζει κατωφλίωση (posterize) και στη συνέχεια εφαρμόζει συνέλιξη (convolution) για την όξυνση των ακμών.

Αρχιτεκτονική και Κώδικας

Ο κώδικας χωρίζεται σε δύο διακριτά μέρη που επικοινωνούν μέσω του διαύλου PCIe/AXI:

1. Host Code (OpenCL - host.cpp):

- Δεσμεύει μνήμη στον επεξεργαστή (RAM) και στον επιταχυντή (Global Memory).
- Αρχικοποιεί τα δεδομένα και εκτελεί μια Software Reference υλοποίηση (Golden Standard) για επαλήθευση.
- Ρυθμίζει τα ορίσματα του πυρήνα (Kernel Arguments) και δίνει την εντολή εκκίνησης μέσω του `cl::CommandQueue`.
- Μεταφέρει τα αποτελέσματα πίσω στον Host και τα συγκρίνει με τη Software υλοποίηση.

2. Kernel Code (HLS C++ - krnl_image_diff.cpp):

- Υλοποιεί τη λογική επεξεργασίας στο FPGA.
- Χρησιμοποιεί κατάλληλα Pragmas για τη βελτιστοποίηση της απόδοσης και τη διασύνδεση με το σύστημα μνήμης.

Τεχνικές Υλοποίησης και Βελτιστοποίηση

Για την αποδοτική υλοποίηση του αλγορίθμου και την αντιμετώπιση των περιορισμών μνήμης (BRAM), εφαρμόστηκαν οι εξής τεχνικές:

i) Tiling (Τμηματοποίηση Εικόνας)

Επειδή η πλήρης αποθήκευση εικόνων πολύ μεγάλων διαστάσεων (π.χ. 4K) στην BRAM είναι αδύνατη, η εικόνα χωρίστηκε σε οριζόντιες λωρίδες (Tiles) ύψους 64 γραμμών (TILE_HEIGHT=64). Ο πυρήνας επεξεργάζεται την εικόνα τμηματικά, φορτώνοντας κάθε φορά μόνο τα απαραίτητα δεδομένα στην τοπική μνήμη.

Η συγκεκριμένη αρχιτεκτονική τμηματοποίησης (Strip Tiling γνωστή και ως Line Buffering) επιλέχθηκε έναντι της τμηματοποίησης σε δισδιάστατα μπλοκ (Block Tiling) για τη βελτιστοποίηση της απόδοσης της μνήμης DDR. Δεδομένου ότι η εφαρμογή είναι Memory Bound, κρίθηκε απαραίτητη η διατήρηση του πλήρους πλάτους (Width) σε κάθε μεταφορά. Αυτό επιτρέπει στον ελεγκτή μνήμης να εκτελεί μακροσκελή Burst Transactions (π.χ. 4096 συνεχόμενα pixels), ελαχιστοποιώντας την επίπτωση του Latency (Start-up penalty) που θα εισήγαγε η συχνή αλλαγή διευθύνσεων σε μια προσέγγιση μικρών μπλοκ.

ii) Διαχείριση Halo (Overlap)

Λόγω της φύσης του φίλτρου 3×3 , κάθε pixel απαιτεί πρόσβαση στους γείτονές του. Στα όρια των Tiles, αυτό δημιουργεί πρόβλημα εξάρτησης δεδομένων.

- Λύση: Για κάθε Tile 64 γραμμών, διαβάζονται από τη μνήμη DDR 66 γραμμές (64 δεδομένα + 1 γραμμή "Halo" πάνω + 1 γραμμή "Halo" κάτω).
- Αυτό επιτρέπει τον σωστό υπολογισμό της συνέλιξης στα άκρα του Tile χωρίς σφάλματα (artifacts).

iii) Διαχείριση Ακριανών Τιμών (Boundary Conditions)

Για τα pixels στα σύνορα της εικόνας (π.χ. $i = 0$, $i = \text{HEIGHT} - 1$, $j = 0$, $j = \text{WIDTH} - 1$, ακολουθήσαμε την πολιτική της αντιγραφής της τιμής από το ενδιάμεσο στάδιο (Posterize), δηλαδή $C_filt[i][j] = C[i][j]$. Η επιλογή αυτή τηρήθηκε αυστηρά τόσο στον κώδικα του Host (sw_reference) όσο και στον HLS κώδικα του επιταχυντή, εξασφαλίζοντας την ταύτιση των αποτελεσμάτων.

iv) Χρήση Pragmas

- `#pragma HLS INTERFACE m_axi`: Ορίζει τις θύρες εισόδου/εξόδου ως AXI Master για γρήγορη μεταφορά δεδομένων (Burst Mode) από/προς την DDR.
- `#pragma HLS INTERFACE s_axilite`: Δημιουργεί καταχωρητές ελέγχου (Control Registers) για την επικοινωνία με τον Host.
- `#pragma HLS PIPELINE II=1`: Εφαρμόστηκε στα loops ανάγνωσης, υπολογισμού και εγγραφής για να επιτευχθεί ρυθμοαπόδοση (throughput) 1 pixel/cycle.
- `#pragma HLS UNROLL factor=8`
- `#pragma HLS ARRAY_PARTITION variable cyclic factor=8 dim=2`

Επαλήθευση & Αποτελέσματα (Emulation)

i) Software Emulation

Η λειτουργική ορθότητα του αλγορίθμου επιβεβαιώθηκε μέσω Software Emulation. Μεταφέρει τα αποτελέσματα πίσω στον Host και εκτελεί στοιχειομετρική σύγκριση (pixel-by-pixel verification) μεταξύ της εξόδου του FPGA και της Software Reference υλοποίησης. Εάν και μόνο εάν όλα τα pixels ταυτίζονται, τυπώνει το μήνυμα 'TEST PASSED'.

ii) Hardware Emulation

Η προσομοίωση υλικού εκτελέστηκε για εικόνα διαστάσεων 128×128 . Δοκιμάστηκαν διάφορες διαστάσεις των tiles καθώς και συνδυασμός loop Unrolling & Array Partitioning με factor=8, καθώς αυτή η τιμή φάνηκε να έχει τη μεγαλύτερη επίπτωση στο πρώτο εργαστήριο. Μέσω του Vitis Analyzer, καταγράφηκαν τα ακόλουθα στοιχεία απόδοσης:

Parameters	Values	Kernel Execution Time (ms)	Top Data Transfer (MB)	Kernel Transfer (MB)	Host (MB)	Transfer
TILE	32	0.385	0.203		0.196	
TILE	64	0.375	0.199		0.196	
TILE	128	0.373	0.197		0.196	
Loop Unrolling & Array Partitioning	factor=8	0.401	0.197		0.196	

Τα αποτελέσματα δείχνουν ξεκάθαρα πως η διαχείριση της μνήμης παίζει τον κυρίαρχο ρόλο στην απόδοση. Αυξάνοντας το TILE_HEIGHT από 32 σε 128, ο όγκος των μεταφερόμενων δεδομένων μειώθηκε από 0.203 MB σε 0.197 MB, καθώς τα μεγαλύτερα tiles μειώνουν το λεγόμενο "halo overhead" (λιγότερες επαναλήψεις ανάγνωσης των ίδιων γραμμών). Αυτή η βελτιστοποίηση οδήγησε και σε μικρή μείωση του χρόνου εκτέλεσης στα 0.373 ms.

Αντίθετα, η προσθήκη επιθετικής παραλληλοποίησης (Unroll factor=8) δεν πρόσφερε καμία ουσιαστική βελτίωση. Αντιθέτως ο χρόνος εκτέλεσης αυξήθηκε στα 0.401 ms. Το γεγονός αυτό αποτελεί αδιάσειστη απόδειξη ότι η εφαρμογή είναι Memory Bound: το FPGA περιμένει τα δεδομένα από τη μνήμη και, όσο γρήγορα κι αν μπορεί να εκτελέσει τις πράξεις (Unrolling), η συνολική ταχύτητα περιορίζεται αποκλειστικά από τον ρυθμό μεταφοράς της DDR. Επιπλέον, το Profile Summary δείχνει Avg Bytes per Transfer = 60.0 bytes. Δεδομένου ότι το εύρος του διαύλου AXI στον Alveo U200 είναι 512-bit (64 bytes), η υλοποίηση επιτυγχάνει 94% Bus Utilization ανά κύκλο (beat), επιβεβαιώνοντας την επιτυχή ενεργοποίηση των Burst Transfers.

4. Συμπεράσματα

Στο εργαστήριο αυτό επιτεύχθηκε η πλήρης υλοποίηση ενός συστήματος επιτάχυνσης στο Vitis. Βασικό συμπέρασμα ήταν η αναγκαιότητα διαχείρισης της ιεραρχίας μνήμης (Global vs Local Memory).

Η τεχνική Tiling με Halo αποδείχθηκε κρίσιμη για την ορθή εφαρμογή φίλτρων συνέλιξης σε υλικό, καθώς επιτρέπει την επεξεργασία εικόνων απεριόριστου μεγέθους διατηρώντας τη χρήση πόρων (BRAM) σταθερή και χαμηλή. Η χρήση του Vitis Analyzer επιβεβαίωσε ότι η σχεδίαση αξιοποιεί αποδοτικά το bandwidth μέσω των burst transfers που εξασφαλίζει το AXI interface.